

Vi IMproved Editor

NCSC101

Prof. Pranay Kumar Saha

August 06, 2025





What is Vim?

- A modal, keyboard-first text editor available on nearly every Unix-like system.
- Extremely fast once you learn motions and operators.
- Highly configurable: mappings, plugins, scripting.
- Works great over SSH; ideal for servers and quick edits.



Why Modal Editing?

- Separate **modes** for **navigation** and **insertion**.
- Fingers stay on home row; minimal mouse usage.
- Compose edits: **operator + motion** (e.g., **daw** = delete a word).
- Repeat with **.** and count with numbers: **3dw**, **5j**.



Modes You Need

Normal Navigate, run commands (default on open).

Insert Type text (**i**, **a**, **o**, ...; leave with **Esc**).

Visual Select text (**v** charwise, **V** linewise, **Ctrl-V** block).

Command-line Ex commands (**:w**, **:q**, **:s/foo/bar/g**).



Entering Insert Mode

- i** insert before cursor
- a** append after cursor
- I** insert at line start
- A** append at line end
- o** open new line below
- O** open new line above

Leave insert mode with **Esc**.



Basic Motions

Key	Action	Tip
h j k l	left, down, up, right	stay on home row
w / b / e	next / prev / end of word	use W/B/E for WORD
o / ^ / \$	bol / first non-space / eol	fast line navigation
gg / G	file start / end	combine with counts: 50G
Ctrl-f / Ctrl-b	page down / up	screen-wise moves
:n	Jump to line number n	:50
* bol: Beginning of Line and eol : End of Line		



Search Navigation

- **/pattern** forward, **?pattern** backward.
- **n** next match, **N** previous match.
- ***** word under cursor forward, **#** backward.
- Set **ignorecase** and **smartcase** for smarter matches.
set ignorecase



Operators + Motions

- Delete: **d** (e.g., **dw**, **d\$**, **dd**).
- Change (delete then insert): **c** (e.g., **cw**, **ci**).
This is the “change” operator in Vim. It deletes text and then immediately puts you into Insert mode, allowing you to type in new text.
- Yank (copy): **y** (e.g., **yw**, **yy**).
- Put (paste): **p**: put after /**P**: put before.

Combine with counts: **3dw**, **2yy**, **5>>**.



Undo/Redo/Repeat

u	undo
Ctrl-r	redo
.	repeat last change



Yanks & Registers

- for Yanking: **yy**
- Default register: unnamed ("").
- Named registers: "**a..**"z (e.g., "**ayy** to yank line to **a**").



Vim Indentation: Normal Mode

Shift (indent) operators

>>	Shift current line right by one shiftwidth .
<<	Shift current line left by one shiftwidth .
[count]>>	Shift <i>count</i> lines right (starting at cursor).
[count]<<	Shift <i>count</i> lines left.
:1,20>>	Shift a line <i>range</i> right (ex command).
:1,20<<	Shift a line <i>range</i> left.



Substitution Basics

`:[range]s/{pattern}/{replacement}/{flags}`

- **range:** Which lines to act on
 - `.` = current line, `$` = last line
 - `1,20` = lines 1 through 20
 - `.,$` = from current line to end
 - `%` = whole file (shorthand for `1,$`)
- **pattern:** Regex to find (Vim regex)
- **replacement:** What to put instead
- **flags:** Behavior tweaks



Substitution Basics

<code>:%s/old/new/</code>	First match per line only (whole file)
<code>:%s/old/new/g</code>	Replace all matches per line (whole file)
<code>.,,\$s/foo/bar/g</code>	From current line to end, replace all
<code>:1,20s/^/# /</code>	Comment lines 1–20 by adding # at start



Vim Command: Commenting Multiple Lines

Command: `:1,20s/^/# /`

Explanation

Part	Meaning
<code>:</code>	Enters Vim's command-line mode.
<code>1,20</code>	Specifies the range of lines: from line 1 to line 20 (inclusive).
<code>s</code>	The <i>substitute</i> command (search and replace).
<code>^</code>	Regular expression anchor for “start of the line”.
<code>#</code>	Replacement text: hash (#) followed by a space.
(no flags)	Applies the substitution once per line (start only).



Vim :substitute Command Options

Syntax: `:[range]s/{pattern}/{replacement}/{flags}`

Common Flags

Flag	Meaning
c	Confirm before each replacement. Prompts: (y/n/a/q).
i	Case-insensitive match (e.g., word matches Word , WORD).
I	Case-sensitive match (overrides ignorecase setting).

Examples

`:%s/foo/bar/gc` " Replace all 'foo' with 'bar', ask each time
`:%s/foo/bar/gi` " Replace all 'foo' with 'bar', ignore case
`:%s/foo/bar/gI` " Replace only exact-case 'foo' with 'bar'



Command Option c actions

Key	Action
y	Yes —replace this match, continue.
n	No —skip this match, continue.
a	All —replace this and all remaining matches (no more prompts).
q	Quit —stop substituting; leave the rest unchanged.



Visual Modes

- Character: **v**
- Line: **V**
- Block: **Ctrl-v** (great for columns)
- After selecting: apply operator (**d**, **y**, **>** etc.).



Open, Save, Quit

:e file	edit file
:w	write/save
:q, :q!	quit (force)
:wq or ZZ	write and quit
:sav newfile	save as



External Commands & Filters

<code>:r !ls</code>	" read output of ls into buffer"
<code>:.! sort</code>	" replace current line with its sorted version"
<code>:.! make</code>	" run shell command"



Running Shell Commands in Vim

Vim can execute shell commands directly and use their output in the buffer.

Syntax

- **:r !cmd** —inserts output of **cmd** into buffer.
- **:[range]!cmd** —sends lines in **[range]** to **cmd** and replaces them with its output.
- **!:cmd** —runs **cmd** in the shell, shows output temporarily.

Tip

:<,> is the range for the last visual selection; **:.** is the current line; **^** in regex means “start of line” .



Help System

`:help` open help
`:help word` specific topic (e.g., `:h cw`)



Vi IMproved Editor

Thank You for Listening!